Parcial de Programación III

Universo Marvel

Fecha límite de entrega: miércoles 13 de mayo

Se solicita modelar un sistema de gestión de personajes y películas del universo Marvel aplicando los conceptos de herencia, clases abstractas, interfaces y relaciones entre objetos en Java. El sistema deberá gestionar Héroes, Villanos y Películas en memoria, representando las relaciones entre ellos de forma orientada a objetos.

Parte 1 — Clase abstracta Personaje

Crear la clase padre, que debe ser abstracta, en el paquete entidad y que debe tener lo siguiente:

1. **Atributos:**
   - id de tipo entero
   - nombre de tipo texto (ej: “Clark Kent”)
   - alias de tipo texto (ej: “Superman”)
   - tipoDePoder de tipo texto (ej: "Magia", "Tecnología", "Fuerza", "Mutante")
   - anioCreacion de tipo entero
2. **Métodos:**
   - **Constructores:** uno vacío y otro con parámetros.
   - **Getters y Setters** de todos los atributos.
   - getAliasCompleto() que debe retornar un texto como: nombre + " '" + alias + "'". Ejemplo: "Clark Kent alias 'Superman'".
   - calcularAntiguedad() que debe retornar el resultado de restar anioCreacion al año actual.
   - mostrarPerfil() que se debe declarar como **abstracto**, para que cada subclase lo implemente con su propia información.

Parte 2 — Clase Heroe

Crear la clase Heroe en el paquete entidad que:

- **Herede** de Personaje.
- **Implemente** la interfaz IHeroe (Esto se hará a lo último, dado que aún no está creada la interfaz).

Además, debe agregarse lo siguiente:

1. **Atributos:**
   - Un atributo propio llamado equipo que será de tipo String (ej: "Avengers", "X-Men", "Guardianes de la Galaxia").
2. **Métodos:**
   a. Propios:
      - **Constructores:** uno vacío y otro con parámetros. Recordá que está heredando de Personaje.
      - Getter y Setter de equipo.

   b. De la clase Padre:
      - Sobreescribir los métodos de la clase Padre: getAliasCompleto() y calcularAntiguedad(). Recordar el uso de la palabra clave super para heredar.
      - Sobreescribir el método mostrarPerfil(), donde debe imprimir: id, alias completo, tipo de poder, antigüedad y, además, el equipo al que pertenece.

   c. De la interfaz: una vez que armes la interfaz IHeroe, implementá los siguientes métodos

      Nota: Primero crear una lista de tipo Heroe que va a simular una base de datos en local.

      - guardar() que agrega el héroe a la lista (validando que no sea null).
      - obtenerTodos() que retorna la lista completa.
      - obtenerPorId() que busca por id con un for, retorna null si no encuentra.

Parte 3 — Interfaz IHeroe

Crear la interfaz IHeroe en el paquete interfaz con el siguiente contrato:

- Un procedimiento, llamado guardar, que no retorna nada y recibe como parámetro un objeto de tipo Heroe.
- Un método que retorna una List de tipo Heroe, llamado obtenerTodos.
- Un método que retorna un objeto de tipo Heroe, llamado obtenerPorId, que debe recibir por parámetro un int idHeroe.

Parte 4 — Clase Villano

Crear la clase Villano en el paquete entidad que:

- **Herede** de Personaje.
- **Implemente** la interfaz IVillano (Esto se hará a lo último, dado que aún no está creada la interfaz).

Además, debe agregarse lo siguiente:

1. **Atributos:**
   - Un atributo propio llamado planMalvado que será de tipo String (ej: "Dominar el mundo", "Destruir Asgard", "Conseguir las Gemas del Infinito").
2. **Métodos:**
   a. Métodos propios:
      - **Constructores:** uno vacío y otro con parámetros. Recordá que está heredando de Personaje.
      - Getter y Setter de planMalvado.

   b. De la clase Padre:
      - Sobreescribir los métodos de la clase Padre: getAliasCompleto() y calcularAntiguedad(). Recordar el uso de la palabra clave super para heredar.
      - Sobreescribir el método mostrarPerfil(), donde debe imprimir: id, alias completo, tipo de poder, antigüedad y, además, el plan malvado.

   c. De la Interfaz: una vez que armes la interfaz IVillano, implementás los siguientes métodos:

      Nota: Primero crear una lista de tipo Villano que va a simular una base de datos en local.
      - guardar() que agrega el villano a la lista (validando que no sea null).
      - obtenerTodos() que retorna la lista completa.
      - obtenerPorId() que busca por id con un for, retorna null si no encuentra.

Parte 5 — Interfaz IVillano

Crear la interfaz IVillano en el paquete interfaz con el siguiente contrato:

- Un procedimiento, llamado guardar, que no retorna nada y recibe como parámetro un objeto de tipo Villano.
- Un método que retorna una List de tipo Villano, llamado obtenerTodos.

- Un método que retorna un objeto de tipo Villano, llamado obtenerPorId, que debe recibir por parámetro un int idVillano.

Parte 6 — Clase Pelicula

Crear la clase Pelicula en el paquete entidad que:

- **Implemente** la interfaz IPelicula (Esto se hará a lo último, dado que aún no está creada la interfaz).

Debe agregarse lo siguiente:

1. **Atributos:**
    - id de tipo entero
    - titulo de tipo texto
    - saga de tipo texto (ej: "Infinity Saga", "Multiverse Saga")
    - anioEstreno de tipo entero
    - duracionMinutos de tipo entero
    - sinopsis de tipo texto
    - villano de tipo Villano: una película tiene un solo villano
    - lista de héroes de tipo Heroe: una película de Marvel tiene varios héroes.
2. **Métodos:**
    a. Métodos Propios:
        - **Constructores:** uno vacío y otro con parámetros.
        - Getter y Setter de todos los atributos.
        - calcularAntiguedad() que debe retornar la diferencia entre el año actual y anioEstreno.
        - esClasica() que debe retornar true si la antigüedad es mayor o igual a 10 años (retorno de tipo boolean).
        - mostrarFicha() que no retorna nada e imprime todos los datos de la película, el alias completo del villano y recorre la lista de héroes mostrando el alias completo de cada uno.

    A tener en cuenta:

    - Al imprimir ¿Es clásica?, debe imprimir **sí** o **no** y NO un booleano como true o false.

- Al imprimir Villano, podemos aprovechar a llamar villano.getAliasCompleto() directamente.
- Al imprimir todos los héroes, primero validar que la lista no es null ni está vacía. Si no lo es, imprimirla con un forEach. Caso contrario, mostrar el aviso: "Sin héroes registrados en el equipo".

b. Métodos de la Interfaz IPelicula: una vez que armes la interfaz, implementás los siguientes métodos:

Nota: Primero crear una lista de tipo Pelicula que va a simular una base de datos en local.

- guardar() que agrega la película a la lista (validando que no sea null).
- obtenerTodos() que retorna la lista completa.
- obtenerPorId() que busca por id con un for, retorna null si no encuentra.
- eliminar() que busca por id y elimina la película de la lista, informando el resultado.

Parte 7 — Interfaz IPelicula

Crear la interfaz IPelicula en el paquete interfaz con el siguiente contrato:

- Un procedimiento, llamado guardar, que no retorna nada y recibe como parámetro un objeto de tipo Pelicula.
- Un método que retorna una List de tipo Pelicula, llamado obtenerTodos().
- Un método que retorna un objeto de tipo Pelicula, llamado obtenerPorId, que debe recibir por parámetro un int idPelicula.
- Un procedimiento, que no retorna nada, llamado eliminar y que recibe como parámetro un int idPelicula.

Parte 8 — Clase Main

En la clase Main se debe demostrar el funcionamiento completo del sistema:

1. Instanciar un objeto servicio vacío para cada entidad (heroeServicio, villanoServicio, peliculaServicio).
2. Crear 3 villanos de tu preferencia y guardarlos usando villanoServicio.guardar().
3. Crear 6 héroes de tu preferencia y guardarlos usando heroeServicio.guardar().
4. Imprimir todos los villanos con mostrarPerfil().
5. Imprimir todos los héroes con mostrarPerfil().

6. Armar 3 listas de héroes (una por película) recuperándolos con heroeServicio.obtenerPorId().
7. Crear 3 películas que te gusten del universo Marvel, asociando cada una a su Villano y su lista de héroes.
8. Guardar las películas y mostrar todas las fichas con mostrarFicha().
9. Buscar una película por ID con obtenerPorId() y mostrar su ficha.
10. Eliminar una película por ID con eliminar() y listar las películas restantes.